20/07/26

Experiment - 01 Study of various Network Commands.

Aim:- To study and practice various network commands used in windows & linux for network configuration.

Windows Commands:

1. arp -a - Display the ARP table with IP & MAC address.
2. hostname - Displays the hostname.
3. ipconfig /all - Displays detailed IP and network configuration.
4. nbtstat -a - Displays active network connection and port information
5. nbtstat - Displays active network Connection for a remote computer.
6. netstat -r - Display the routing table
7. nslookup - Finds the IP address associated with a domain name.
8. pathping - Traces the network path and check packet lost at each loop.

5. netstat: (Network statics) netstat display a variety of statistics about a computer active TCP connection:

O/P: Active Connection

Proto	local address	foreign address	Stax
TCP	10.16.56.50.4	4.213.25.242	Established

6. nsloopup: It is used to display the DNS details such as IP address of a particular computer.

O/P: Server: Unknown

address: 10.16.56.155

7. pathping: pathping is a unique window and its basically a combination of the ping and Tracert Commands.

cmd/: pathing ping -g google.com

O/P: Tracing route to google.com [142.250.67.14] over a minimum of 30hps:

Trace complete.

8. Route: route command is used to show the IP routing table. It is primarily used to setup static route to specific

O/P: route [-f] [-p] [-4] [-6] command [destination]

Cmd: Route print

B... ca 00 27 00 0d ... virtual box

Linux Commands:

1. IP Commands:

O/P: cmd: ip address show displays all ip addresses of network interfaces.

eth0: inet 192.168.1.10/24

2. IP address to an Interface:

O/P: ip address add 192.168.1.254/24 dev enp503

ip address added Sucessfully.

3. ip link set ethoup - enables the network interface

O/P: eth0 static up

4. ip link set eth0 down - distables the network interface

O/P: eth0 static down

5. ip link set promic on - Enables pronicous mode

O/P: Promoicous mode enabled.

6. ip route add default via 192.168.1.254

20. nmcli connection show - list all network connections

O/P: wired connection 1

:

21. nmcli connection modify "wired connection 1" modifies an existing connection.

22. nmcli connection show displays connection details.

O/P: ipv4.method auto.

23. nmcli connection modify "wired connection" ipv4 method auto configures DHCP

O/P: IPv4 method set to auto.

25. nmcli connection modify "wired connection" ipv4 method manual ipv4 address 192.0.2.1/24 ip4 gateway 192.02.254 ipv4.dns

O/P: 192.0.2.200 - configures a static IPv4 add

IPv4 settings updated.

Result: Thus various windows and linux command

10/08/24

Exp: 4

Setup and Configure a LAN using a switch and ethernet cables in lab

Aim:

To setup and configure a LAN using a switch and ethernet cables in your lab.

What is LAN?

LAN stands for Local Area Network which is a group of computers and connected devices linked together in a small physical location like a school, home or office building.

How to Setup LAN?

Step 1: Connect the hardware

* place the router in the central location near a power outlet.
* Connect your modem to the router WAN. using ethernet cable.

Step 2: Connect your Devices:

Wired Setup: plug one end of ethernet cable into your computers network port and the numbered LAN ports on the router.

Step 4: Configure the Router

* Type your IP address 192.168.1.1 into the address and press enter.
* Log in using the default username and password printed on the router.

Step 4: Check the Connectivity:

* Between switch and other machine by using PING command in the command prompt of the device.

Step 5: Confirm the Connectivity

Try to access the shared folder from other computers of the networks.

print ("Sender, Hamming code:", "join map(str, code)).

Reciever Side:

- Reciever the Hamming code from the sender.
- Enter the recieved code
- calculate the parity check values.
- find the error position using parity values.

CODE:

```
r= list (map (int, input (" Enter recieved 7 bits " ) ) )

r1= r[0] ^ r[2] ^ r[4] ^ r[6]

r2 = r[1] ^ r[2] ^ r[5] ^ r[6]

r3 = r[3] ^ r[4] ^ r[5] ^ r[6]

error = r1 * 1 + r2 * 2 + r4 * 4

if error == 0
    print ("No error")
else
    print (" Error at position :", error)
    r[error-1] ^ = 1

print (" Corrected code:", .join (map (str, recieved
```

Input:

Enter 4 data bits : 1011

Sender Hamming code: 0110011

Recieved 7 bits : 0010011

O/P:

Error at position : 2

Corrected position : 0110011

Result:

Error detection and Error correction using hamming code. was executed

28/9/24

Result: The classless IP subnetting was Sucessfully Implemented using CISCO packet TRACER.

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
12.	29/9/26	End to End Communication at Trans. layer	29	(10) 29/9
13.	29/9/26	Implementation your own ping command	32	(9) 29/9
14	01/10/26	Packet Sniffing using raw socket.	36	(9) 01/10/26

INDEX

NAME: S. Rajeshwar STD.: 3rd Yr SEC.: _____ ROLL NO.: 1413 SUB.: Computer Network Lab

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
1.	20/07/26	Exp:01: Study of various Network Commands	01	9
2.	28/7/26	Exp:02: Study of different type of Network Cables	04	9
3.	10/08/26	Exp:03: Cisco Packet Tracer	07	9
4.	10/08/26	Exp:04: Setup & Config a LAN using a Switch and ethernet cables in Lab	09	9
5.	18/08/26	Wire Shark	013	9
6.	07/09/26	Error detected & error correction	16	9
7.	08/09/26	Flow Control using Sliding window protocol	19	9
8.	22/09/26	NAP to Discover live host using Nmap Scans on the tryhackme platform	23	9
9.	28/09/26	Implementation of Subnetting in cisco packet Tracer Simulation.	26	9

GKC1025602 G

Ex 01/10/26

25

Packet Sniffing Using RAW Sockets

Aim:

To write a program using Raw Sockets to capture and display network packets.

Algorithm:

1. Create a RAW socket
2. Bind the socket to the network interface.
3. Recieve packet from the network.
4. Extract the IP header information
5. Display the source IP, Destination IP and protocol.

Code:

```
Import Socket
S= Socket. Sock C Socket. AE_INET, Socket
                Sock =RAW, Socket. IPPROTO_IP)
S. bind C "0.0.0.0', 0);
S. set Sockopt C Socket . IPPROTO _IP, Socket . IP
                - HDR INCI, 1)
Print C" Packet Sniffer started..." )
while True:
    data . add = S. recv from (65535)
    print C"packet recieved from:", add(0)
```

Input:

Network packet

Output:

```
Packed Sniffer Stacked. . . .
packed recieved from: 192.168.1.5
packed recieved from: 192.168.1.1
packet recieved from: 192.20.183.14
```

Result:

The Raw Socket Sniffing Program. was Sucessfully Implemented and network packets was captured and displayed.

1/10/26

Exp: 13

29/9/26

Implement your own ping program

Aim:

To Implement a ping program using python to check the connectivity between source and destination host.

Algorithm:

1. Get the destination IP address from user.
2. Send the request to destination.
3. wait for response.
4. Calculate the response time.
5. Display whether the host is reachable.
6. Repeat the process for few request

Code:

```
import os
import time
host = input("Enter host:")
for i in range(4):
    start = time.time()
    result = os.system("ping -n 1 "
        + host + ">null")
    end = time.time()
    if result==0:
        print("Reply from", host,
            "Time =", round((
            end-start)*1000,2),"ms")
    else:
        print("Request timed out")
    time.sleep(1)
```

Input:

Enter host : 127.00.1

Output:

Reply from 127.0.0.1 Time = 123 ms

Reply from 127.0.0.1 Time = 0.98 ms

29/9/26

Result:

The ping program was suasfully Studied and Implemented.

b). Implement that client

TCP/ IP sockets.

Aim:-

To Implement a chat client Server application using TCP sockets.

Code: (Server)

```
Import socket

S= socket. socket()

S.bind (("Local host", 5000))

S.listen()

c, a = S.accept()

while True:
    msg = c.recv(1024). decode()
    print ("client:", msg)
    if msg == "byte":
        break
    reply = input("Server!")
    c. send (reply . encode())

c-close()
```

Client:

```
Import socket

S= socket. socket()

S= connect (("local host", 5000))

while true:
    msg = Input ("Client :")
    S . send ( msg. encode())
    if msg == "bye"
        break
    print ("Server!", S.recv (1024).decode());

S.close():
```

Input:

Client: Hello

Server: Hi

Client: Bye

Output:

Client: Hello

Server: Hi

Result:

The Chat client Server application were Sucessfully studied and implemented using TCP Sockets.

Exp No: 12

End to End Communication of Transport layer.

29/9/26

Eg. Implement Echo client Server using TCP sockets.

Aim: To implement an Echo server application using TCP Sockets.

Code: (Echo Server)

```
Import socket
    s = socket.socket()
    s.bind ("localhost", 5000)
    s.listen()
    c, add = s.accept()
msg = c.recv(1024).decode()
print("client", msg)
c.send(msg.encode())
c.close()
s.close()
```

Echo Client:

```
Import socket
s = socket.socket()
s.connect("local host", 5000)
msg = Input("Enter message:")
s.send(msg.encode())
print("Echo:", s.recv(1024).decode())
s.close()
```

I/P:

Enter message : Hello Server

O/P:

Server

Client Connected

Recieved : Hello Server

Client

Echo from server : Hello Server.

Result:

The Echo client Server using TCP Socket was studied and executed Sucessfully.

29/9/26

22/9/24

Result:

Using TryHackMe platform we learned and executed to discover the live host on nmap scans (ARP, ICMP, TCMP)

Exp.8 NMAP to discover live hosts using NMAP Scans on the tryhackme platform.

22/08/24

Aim:-

To discover live hosts using NMAP Scans (ARP, ICMP, TCP/UDP) on the TRYHACKME platform

Roomlink:- https://tryhackme.com/room/nmapol

Procedure:

TASK 1:

- ARP Scan: This Scan uses ARP request to discover live hosts.
- ICMP Scan: This Scan uses ICMP request to identify live host.
- TCP/UDP live PING Scan: This Scan sends packets to TCP ports and UDP ports to determine all live hosts.

TASK:2 :- Subnetworks:

Network segment:- It is a collection of computers linked together through a common "medium".

Reciever:

```
import time
import os
expected = 0
while True:
    if not os.path.exist("sender_buffer"):
        continue
    with open("sender_buffer") as f:
        frames = f.readlines()
    for x in frames:
        n = int(x.split(":")[0])
        if n == expected:
            expected += 1
            print("Recieved:", n)
            with open("Reciever_buffer", "w") as f:
                f.write("ACK" + str(expected))
        else
            with open("Reciever_buffer", "w") as f:
                f.write("NACK:" + str(expected))
            break.
    time.sleep(1).
```

Exp: 7:

Flow Control using Sliding Window protocol.

08/09/24

Aim:

Write a program to implement flow control at data link layer using Sliding window protocol.

Sender:

- Get the window size from user
- Get the text message
- Divide the messages into frames.
- Send to Sender_buffer
- wait for acknowledgement
- Read Reciever_Buffer
- If ACK is recieved send the next
- If NACK is recieved retransmit the frame.

Reciever:

- Read frames from Sender_buffer
- check the frame number.
- If the frame number is expected

Result: Wire Shark in packet capture tool experiment are studied and executed successfully.

- Click start capture.
- Search DNS packets in search box.

Create a filter to display only DHCP packets and inspect the packets

- Select Local Area Connection in wireshark.
- go to capture option
- Select stop capture automatically after 100 packets.
- Then click start capture.
- Search DHCP packets in searchbox.

8/12

Exp No: 6

Wire Shark:

Aim:

To filter, Capture, view, packets in wire shark tool.

Procedure:

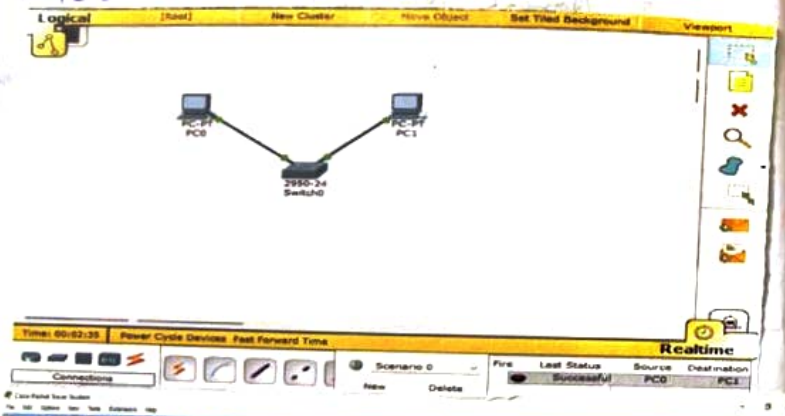
- Select local Area Connection in wireshark
- To capture option
- Select Stop capture automatically after 100 packets.
- Then click shark capture.
- Search TCP packets in Search bar.
- Save the packets.

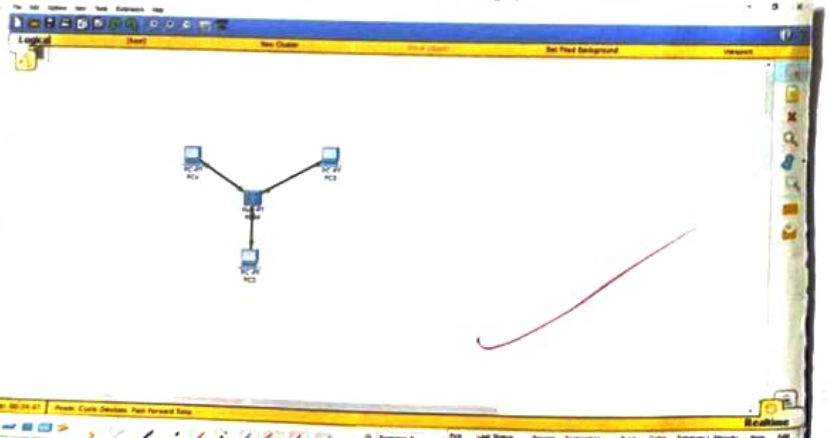
1. Create a filter to display only TCP/UDP packets Inspect the packets and provide the flow graph.

- Select LAN connection in wire shark.
- To capture option.
- Select stop capture automatically after 100 packets.
- Then click start capture.
- Save the packet.

3. Networking Switch:

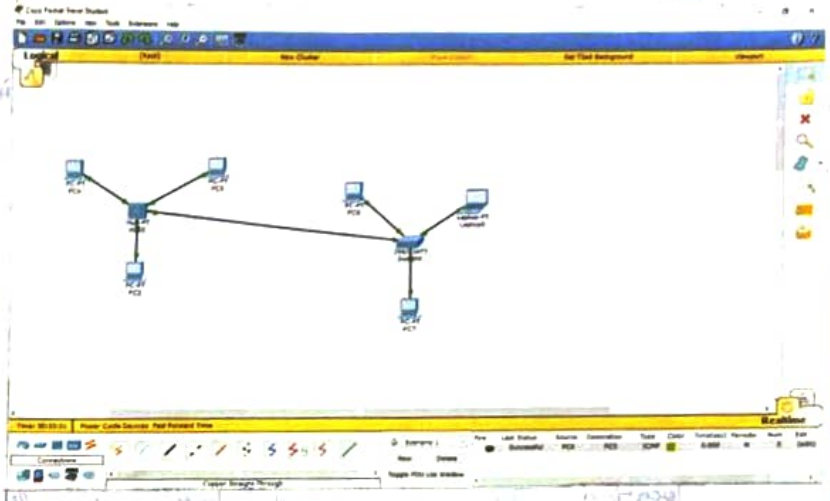
- Connects Computers and forwards data to the required destination.
- A network was created by connecting 3 pcs to a Switch using copper straight through cables. IP addresses were assigned to all pcs cable IP addresses were assigned to all pcs and connectivity was tested using the ping command. The Switch forwarded the packet to Command the Switch. allowing Sucessful communication between the PCs.





4. Hub & Switch:

- Both hub and Switch were used in the network the packet forwarding behaviour of both devices was observed and compared.



Result:

The peer to peer, Hub & Switch networks were successfully designed and their Communication

Exp:
03

Aim:-

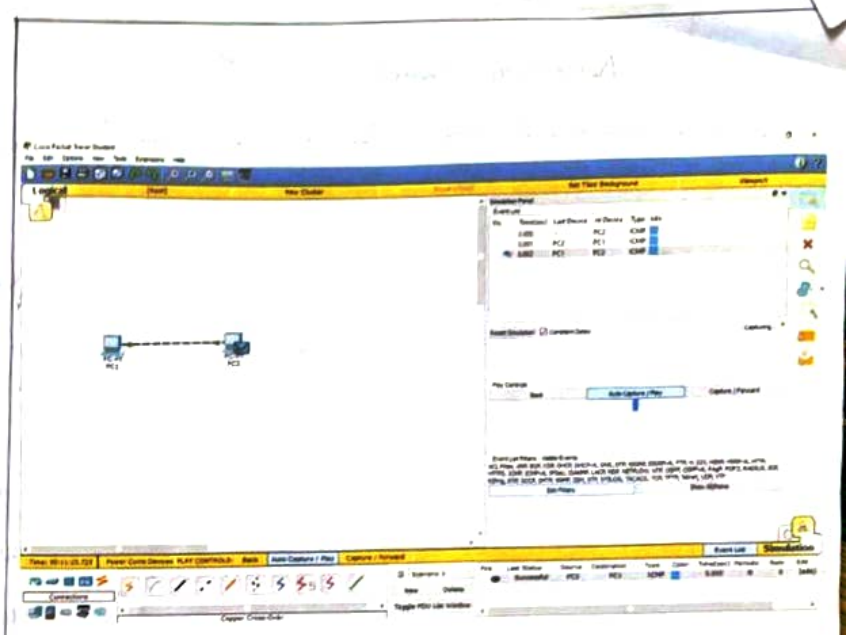
To Study the cisco packet tracer simulation tool, understand its user interface, design a simple network and analyse the behaviour of hub, switch and peer to peer.

Ciscopacket tracer:-

Cisco packet Tracer is a network Simulation and modelling tool. It allows us to create and test computer networks without using real networking equipment we can add PCs, switches hubs, routes and other devices and observe how packets travel between them.

1. Peer to Peer:-

- Two computers communicate directly
- A peer to peer network was created by connecting 2 pcs directly. IP address were assigned to both PCs in the same network The Connectivity between the 2 PCs was tested using the ping Command Successful. Two PC communicate each other



2. Network using Hub:-

- Connected multiple Computers if forward recieved data to all ports.
- A network was created by connecting 3 pcs to a hub using copper straight Through cables. IP addresses were assigned to all the pcs. when data was sent from one PC to another, the hub forward the packet to all connected ports. The destination PC recieved the packet of sucessful Communication was observed.

STP	Category 6,6a	10Gbps	Adv: • Shielded • Faster • less susceptible to noise and interference	Gigabit ethern 10G ethernet Widely used in data centres.
SSTP	Category 7	10Gbps	Dis-adv: • Expensive • greater installation effort	Gigabit ethern 10G ethernet (100m)
Coaxial cable	RG-6 RG-59 RG-11	10-100 mbps	Adv: • Higher bandwidth • Immune to interf -erence • low loss bandwidth • Versatile Disadv: • Limited distance • Cost • Size is bulky	Speed of Signal is 500 Television an -rk high Spe -ed internet Connection.

28/7/26

Experiment: 2 Study of different types of Network cables

Aim:- To study different types of network cables.

Different types of cables used in networking are:

1. Unshielded Twisted Pair (UTP) cable
2. Shielded Twisted Pair (STP) cable
3. Coaxial Cable
4. Fibre optic Cable.

Cable Type	Category	Maximum Data Transmission	Adv/ Dis-Adv	Application/Use
UTP	Category 3	10bps	Adv: • cheaper • Easy to install as they have smaller overall diameter. Dis-adv: • More prone to electro magnetic interface.	10 base-T ethernet
	Category 5	Upto 100 mbps		Fast ethernet Gigabit ethernet
	Category 5e	1Gbps		Fast ethern Gigabit etherne

7. ip route get 10.10.1.4 - shows the route destination ip.

O/P: 10.10.1.4 via 192.168.1.254 dev

8. ifconfig

O/P: Displays network interface configuration

9. mtr:

mtr google.com shows route statistics

O/P:

Host: localhost Avg bot curve.

10. mtr -g google.com

shows numeric ip address

O/P:

142.250.183.286.

11. mtr -b gogle.com shows both username and address

O/P: google.com (142.250.183.206)

12. mtr -c 10 google.com sends only 10 packets

Op: Sent 10 packets.

13. tcpdump:

1. dnf install g- dump -install tcpdump
2. tcpdump -D- list available capture interface